Multer image upload implementation

Overview

The CMS now sends selected image files to the Express backend as multipart/form-data. Multer validates and stores the images in backend/uploads/. The API returns a stable URL such as /uploads/<generated-name>.png; the About, Pulse & Page, and site-section content APIs save that URL alongside the rest of the CMS content.

Existing image URL and site-path entry is still supported.

Request flow

1. An administrator selects a JPG, JPEG, PNG, or WEBP file in an image field.
2. The shared frontend helper posts the file under the image form field to POST /api/uploads/image.
3. Multer checks the image MIME type, extension, and 5 MB file-size limit, then saves the file using a generated UUID filename.
4. The upload route checks the file signature and removes the file if its bytes do not match the declared image type.
5. The API returns { "url": "/uploads/<filename>" }. The CMS stores that path in the related About, Pulse, or site-section content.
6. Express serves the saved image at /uploads/<filename>. Vite proxies that path to the backend while developing.

API details

- Upload: POST /api/uploads/image
- Request content type: multipart/form-data
- Form field: image (one file)
- Accepted formats: JPG/JPEG, PNG, WEBP
- Maximum file size: 5 MB
- Success response (201): { "url": "/uploads/<generated-name>.<ext>" }
- Invalid/missing image: JSON error response with HTTP 400
- Oversized image: JSON error response with HTTP 413
- Image delivery: GET /uploads/<generated-name>.<ext>

Files changed

- backend/Middleware/upload.js: creates the upload directory; configures Multer disk storage, safe generated filenames, allowed formats, and limits.
- backend/routes/uploadRoutes.js: adds the upload endpoint and verifies image signatures before exposing a saved file.
- backend/server.js: mounts the upload API and serves persisted uploads; returns clear Multer and request error responses.
- backend/package.json and backend/package-lock.json: add Multer as a backend runtime dependency.
- .gitignore: excludes persisted uploads from source control.
- vite.config.js: proxies /uploads to the backend during local development.
- src/api/uploadApi.js: sends files with FormData and validates API replies.
- src/components/ImageUploadField.jsx and src/components/ImageUploadField.css: shared URL/file inputs, progress, size/type checks, and visible errors.
- src/components/AboutAdmin.jsx: uses the shared upload field for the leadership portrait.
- src/components/PulsePostForm.jsx: uploads Pulse images instead of embedding selected files as base64 data URLs.
- src/components/SiteSectionsAdmin.jsx: connects Showtime, Performance Spaces, Legacy Library, and Culinary Experience image fields to the upload API and prevents saving while uploads are in progress.

Running and verifying

Start the backend from backend/ with npm run dev, and start the frontend from the project root with npm run dev. In the CMS, choose an image and wait for the upload confirmation before saving the section or post. The saved image URL should begin with /uploads/ and load directly in the browser.

The backend upload folder is created automatically when the server starts. Back up backend/uploads/ with the application data because database records refer to the generated files by URL.